



FSEGSFAX

Département Informatique

Pré-Test

Double diplôme
Université de Le Mans

Moyenne 1 ^{ère} année	
Moyenne 2 ^{ème} année	
Moyenne 3 ^{ème} année	

NOM ET PRENOM :

Groupe :

Durée : 1h30

Année Universitaire : 23/24

Documents : non autorisés

► PARTIE 1 : Les Commandes Linux (3.5 pts)

Donnez le nom de la commande linux permettant de : NB : donnez un seul nom, s'il y en a plusieurs	
Se déplacer dans une arborescence	... <code>cd</code>
Lister le contenu du répertoire de travail	... <code>ls</code>
Trouver un document	... <code>find</code>
Chercher un mot dans un flux texte	... <code>grep</code>
Editer un fichier texte en mode console (terminal)	... <code>nano</code>
Afficher le contenu d'un fichier texte en numérotant ses lignes	... <code>cat -n</code>
Lister les processus en cours d'exécution	... <code>ps</code>
Envoyer un signal à un processus	... <code>kill</code>
Quel est l'opérateur utilisé entre les commandes pour une	
Exécution séquentielle	... <code>;</code>
Exécution parallèle	... <code>&</code>
Exécution à condition d'erreur	... <code> </code>
Exécution à condition de succès	... <code>&&</code>
Donnez le symbole utilisé pour rediriger	
La sortie standard	... <code>></code>
La sortie d'erreur	... <code>2></code>

► PARTIE 2 : Compilation C et création du Makefile (5pts)

Q1 : Un projet C passe généralement par 4 étapes de compilation permettant de transformer le code source en un exécutable. Citez-les :

1..processing.....2..compilation.....3. assembly.....4..linking.....

Q2. Quel est l'intérêt de la création d'un Makefile pour un projet C ?

un make file fait un mise a jour automatique d'un fichier quand execute cette make file au lieu d'executer plusieurs commande

Q3. Quel est le rôle d'option -WALL de la commande gcc

donne le niveau de warning

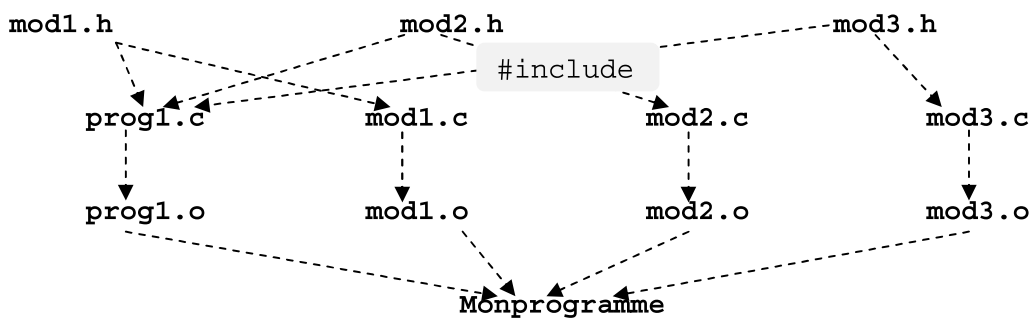
Q4. Que représentent les variables spéciales suivantes dans un fichier Makefile

\$@:..represente le nom du cible

\$^ :..represente toute les dependances

\$< :..represente la 1ère dependance

Q5. Soit un projet C dont le schéma de dépendances entre ses fichiers est le suivant



Le dossier principal du projet est structuré comme suit:

```
. : Module1  Module2  Module3  prog1.c  Makefile
./Module1 : mod1.c mod1.h
./Module2 : mod2.c mod2.h
./Module3 : mod3.c mod3.h
```

Complétez l'écriture du fichier Makefile afin de compiler avec succès le projet C

```
CC=gcc

EXE=Monprogramme

$(EXE) : prog1.o module1/mod1.o module2/mod2.o module3/mod3.o
        $(CC) prog1.o mod1.o mod2.o mod3.o -o $(EXE)

Module1/mod1.o:Module1/mod1.c Module1/mod1.h
        $(CC) -c module1/mod1.c -o $@
```

-I. signifie : chercher les fichiers .h dans le répertoire courant

```
Module2/mod2.o:Module2/mod2.c Module2/mod2.h      $@=module2/mod2.o
$(cc) -c module2/mod2.c -o $@
-----
Module3/mod3.o:Module3/mod3.c Module3/mod3.h      $(cc) -c $< -o $@
$(cc) -c module3/mod3.c -o $@
-----
prog1.o:prog1.c
$(cc) -c prog1.c -o prog1.o -I .mod1.h -I .mod2.h -I .mod3.h
```

► PARTIE 3 : Programmation système UNIX en C (11.5 pts)

Exercice1 (1.5 pts) : Combien de lignes «**Test_DD**» affiche chacun des programmes suivants ?

<pre>#include<stdio.h> #include<unistd.h> #include <sys/wait.h> int main(){ int i ;pid_t pid; for(i=0 ;i<2 ;i++) { pid=fork(); printf("Test_DD\n"); } while(wait(NULL) > 0); exit(0);}</pre>	<pre>#include<stdio.h> #include<unistd.h> #include <sys/wait.h> int main(){ int i ;pid_t pid; for(i=0 ;i<2 ;i++) { pid=fork(); if(pid==0){ printf("Test_DD\n"); break;} } while(wait(NULL) > 0); exit(0);}</pre>	<pre>#include<stdio.h> #include<unistd.h> #include <sys/wait.h> int main(){ int i ;pid_t pid; for(i=0 ;i<2 ;i++) { pid=fork(); if(pid==0) printf("Test_DD\n"); } while(wait(NULL) > 0); exit(0);}</pre>
---	---	--

.....4.....

.....3.....

.....2.....

Exercice 2 (3 pts) : Complétez la fonction main du programme **proc.c** suivant dans le quel un processus père crée N processus fils en parallèle. Chaque processus fils affiche le message suivant: "**Je suis le processus nb=i**" où i est le numéro du processus fils.

On note que N est un entier positif > 1 et il sera saisi en ligne de commande lors du lancement du programme.

Voici un exemple d'exécution du programme **proc.c** sous un environnement linux

```
fsegs@ubuntu:~$ ./proc 4
Je suis le processus nb=1
Je suis le processus nb=2
Je suis le processus nb=3
Je suis le processus nb=4
```

```

int main(int nb_arg , char * tab_arg[] , char * env[]){
int i,N ;pid_t  pid;
if( nb_arg < 2 ){
    fprintf( stderr , "%s : la valeur N est manquante" ,tab_arg[0] );
    exit(-1);}
// Récupération de N
if( sscanf( tab_arg[1] , "%d" , &N)!=1 || N<1){
    fprintf( stderr , "N doit être un entier strictement positif");
    exit(-1);}
// Création de N processus fils

for(i=1;i=N+1;i++)
{
    pid=fork()
    if(pid==0)
    {
        printf("Je suis le processus nb=%d",i);
    }
}

exit (0); }

```

Exercice 3 (7 pts): Complétez le programme **crypte.c** suivant pour qu'il exécute en même temps deux processus :

1) un processus père:

- Demande à l'utilisateur de saisir un message (une chaîne de caractères)
- Envoie le message vers son processus fils via un tube (pipe)
- Attend l'arrivée de son message crypté depuis son processus fils
- Affiche le message crypté.

2) un processus fils:

- Attend l'arrivée du message du processus père et l'affiche sur l'écran
- Crypte le message reçu en ajoutant 3 au code ASCII de chaque caractère formant le message. On note que le code ASCII doit être calculé en termes de modulo 255.
- Envoie le message crypté vers son processus père via un autre tube (pipe)

Voici un exemple d'exécution du programme **crypte.c** sous un environnement linux

```
fsegs@ubuntu:~$ ./crypte
***Processus pere***
Donnez un message: Test
***Processus fils***
Reception du message non crypte: Test
***Processus pere***
Reception du message crypte: Whvw
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define LG_MESSAGE 126
int main()
{
    int i,tube1[2],tube2[2];
    char message[LG_MESSAGE] ;

// création des tubes partagés entre les 2 processus
if(pipe(tube1)||pipe(tube2) )
{
    fprintf( stderr, "erreur de creation des tubes");
    exit(-1);
}

pid_t pid=fork();
if(pid==-1)
{
    fprintf( stderr, "Erreur d'execution de fork");
    exit(-1);
}
else if (pid==0)//processus fils
{
        ..close(tube1[1]).....
        read(tube1[0],message,strlen(message)+1);
        ..close(tube1[0]).....
        close(tube2[1])
        ..read(tube2[0],message,strlen(message)+1);.....
        close(tube2[0])
        .....
        .....
        .....
        .....
        .....
        .....
        .....

```

